TODO LIST & REMAINING SCOPE

Rishta & Rang

Version 1 (MVP) — What Is Done, What Is Left

Prepared 1 September 2026 **Baseline** branch `dev-faizan` @ `39ebd3a` · app `1.0.3` (versionCode 4)
Stack Expo SDK 54 / RN 0.81 · Supabase (Postgres + Auth + Storage + Realtime)
Scope reference *2-in-1 App Feature Roadmap — Version 1 (MVP)*

1. Snapshot

Every V1 feature named in the roadmap exists in the app in some form — nothing is at zero. The gap is that **the half of each feature that needs a second real user** is not built: matching, messaging, the "Move to Rishta" handshake, blocking and reporting all currently operate on one side only. Money and legal compliance are the other two open fronts.

7	7	0	14	18	6
DONE	PARTIAL	NOT STARTED	V1 FEATURES	OPEN TODOS	P0 BLOCKERS

The single blocker behind most of this list

Matches and chat messages are stored per-owner: a `matches` row and every `chat_messages` row carry the sender's own `profile_id`, and RLS restricts reads to `profile_id = auth.uid()`. The recipient can never read either one. The "Move to Rishta" acceptance is a 2.2-second `setTimeout`, stated in the code itself (`src/store/MatchesContext.tsx:199` — "*There's no live backend/second user in this app*"). Until the schema is reshaped to pair rows, the roadmap's core hypothesis cannot be measured at all — and items 4, 6, 7, 11 and 14 below cannot be built on top of it.

2. V1 Feature Checklist

The 14 features the roadmap lists for V1, each against what is actually in the branch today.

#	FEATURE (PER ROADMAP)	STATUS	WHAT IS BUILT	WHAT IS STILL MISSING
UNIVERSAL — BOTH MODES				
1	Onboarding & Intent Setting	DONE	3-step flow (signup → intent + photos → selfie), single intent choice, 2–4 photos enforced, bio, profile rows written before media so a failed upload cannot strand an account.	—

#	FEATURE (PER ROADMAP)	STATUS	WHAT IS BUILT	WHAT IS STILL MISSING
2	Photo Upload & Selfie Verification	PARTIAL	Upload to Supabase Storage via `expo-file-system` raw bytes; selfie + CNIC mandatory; private bucket reachable only by signed URL; `selfieVerified` badge on the profile.	The selfie is stored but **never read back or reviewed** — the badge is granted automatically. CNIC "verification" is a colour / aspect-ratio heuristic (`src/utils/idCardImageCheck.ts`) that sets `cnic_verified = true` on its own; honest as a wrong-upload filter, but it must not be presented as identity verification.
3	Urdu + English UI	DONE	602 keys each in English, Urdu and Roman Urdu; RTL layout support; language choice persisted.	Final sweep only — a handful of strings still fall back to English (item 16).
4	In-App Text Messaging	PARTIAL	Full chat UI, message bubbles, Realtime channel subscribed, image and voice-note kinds, unread flag on the match row.	**One-sided.** `chat_messages` is keyed on the sender's `profile_id` with `from_me` baked in at write time; the other member cannot read the row, and Realtime filters the same way. **No emoji reactions**, which the roadmap names explicitly. No pagination, no delivered / read state.
5	Block & Report	PARTIAL	One-tap block and report UI on profile and chat (`ReportDialog` , `ProfileUtilityBar`); `blocked_users` table and a Blocked Users settings screen.	**Report goes nowhere** — the reason is shown in a toast and discarded; there is no `reports` table and no moderation path. **Block is one-directional**, and `blocked_id` stores the match-row id rather than the person's uuid, so the blocked person still sees you and — once chat is two-way — can still reach you.
6	Basic Match Suggestions	PARTIAL	Filter-based suggestion service over real Supabase profiles with age / city / intent filtering, gender matching and caching.	The deck **tops itself up with mock profiles** whenever fewer than 8 real members match (`DiscoveryContext` → `mockDiscoverProfiles`), so a demo shows fabricated people as if they were users.
DATING MODE				
7	Swipe Discovery	DONE	Tinder-style swipeable deck with gesture handling, action bar, boost sheet, view history and ten sort modes.	—
8	Vibe Tags	DONE	Self-selected tags editable in Edit Profile, shown on the card and detail view, and fed into the compatibility helper.	—
9	'Move to Rishta' Button	PARTIAL	Button in the matched chat, request state on the match row, pending banner, celebration and mode flip in the UI.	**The acceptance is faked.** The request is inserted and then the app answers itself after 2.2 s (`MatchesContext.tsx:212`). The other member is never asked and never sees a request, so the conversion the whole product exists to test produces no real signal.
RISHTA MODE				

#	FEATURE (PER ROADMAP)	STATUS	WHAT IS BUILT	WHAT IS STILL MISSING
10	Basic Matrimonial Profile	DONE	Religion, sect, family background and education captured in `RishtaProfileScreen`, stored on `profiles`, rendered on the rishta listing card and detail sections.	—
11	Rishta Readiness Signal	DONE	Three-state tag (just browsing / ready in a few months / ready now) set on the profile and visible on cards, detail and listings.	—
12	Manual Filters	DONE	Age, city, sect, intent and readiness filters plus verified-only and active-today, through `FilterSheet` / `BrowseSheets`, applied server-side in `discoveryService`.	—
MONETIZATION				
13	Free Tier	PARTIAL	15 likes a day (`DAILY_FREE_LIKES`), persisted per user per date in `daily_likes` ; profile, both modes, matched chat and Urdu UI all free.	The cap is enforced **in app state, against a row the user can write themselves** — trivially bypassed with a direct API call.
14	Single Paid Unlock (Explore+ lite)	PARTIAL	Full paywall screen, plan and trial copy, unlimited likes and "who liked you" gated on the flag.	**No payment of any kind.** "Upgrade" simply sets `isExplorePlus: true` on the user's own row (`ExplorePlusScreen.tsx:87`), and the column is client-writable — the paid tier is free to anyone who taps it.

3. The TODO List

Eighteen open items. **P0** blocks delivery to the client, **P1** blocks a public launch, **P2** is quality and hardening that should still land before handover. "Est." is working days for one developer.

#		TASK	PRI	WHAT HAS TO BE DONE	DONE WHEN	EST.
A · MAKE THE CORE LOOP REAL — NOTHING ELSE CAN BE TRUSTED UNTIL THIS IS DONE						
1	☐	**Mutual matching schema**	P0	New migration `19_matching.sql` : a `likes` table keyed on `(liker_id, target_id, mode)` , and `matches` reshaped to a pair row `(user_a, user_b)` ordered so the pair is unique. RLS: readable by either participant. Migrate the existing one-sided rows; add both to the Realtime publication.	Two accounts see the same match row; a third account sees nothing.	2
2	☐	**Mutual like → match**	P0	A `like_profile(target, mode)` RPC that records the like, checks for the reciprocal one and creates the match atomically, returning whether it matched. Rewire `FavoritesContext` and the swipe handlers onto it.	A likes B, B likes A → both get the match; A alone gets nothing.	1
3	☐	**Two-way messaging**	P0	`chat_messages` re-keyed to `match_id + sender_id` ; RLS becomes "you are a participant of this match"; `from_me` derived at read time. Rewrite `matchesService` and the Realtime subscription to filter on match membership. Per-side unread counters.	A message on device A appears on device B within a second, and unread clears per side.	1
4	☐	**'Move to Rishta' for real**	P0	Delete the `setTimeout` simulation. The request becomes a state on the shared match row; the other member sees a pending banner with Accept / Decline; accepting flips the mode for both sides and notifies both. Gate the request on the rishta profile fields being filled.	The signature feature works between two real accounts and is visible in the data.	1
5	☐	**Celebrate real matches only**	P1	The match celebration currently fires on every right-swipe. Fire it from the RPC's "it matched" result instead.	The celebration appears only when both people liked each other.	0.5
B · SAFETY AND TRUST — THE ROADMAP'S MINIMUM BAR BEFORE ANY PUBLIC TEST						
6	☐	**Reporting that goes somewhere**	P0	A `reports` table (reporter, target, reason, context, free text, status) with insert-only RLS for members. Wire the existing `ReportDialog` in Home and Chat to it. Auto-hide a profile from discovery after N distinct reports. A simple review view for the client in Studio.	A report is a row someone can act on, not a toast.	1
7	☐	**Block hardening**	P1	`blocked_id` becomes the person's uuid, not the match-row id. Enforce in the database: blocked pairs excluded from discovery both ways, message insert refused in either direction, blocked people gone from Explore, likes and matches on both sides.	After a block, neither member can see or reach the other from any surface.	1

#		TASK	PRI	WHAT HAS TO BE DONE	DONE WHEN	EST.
8	☐	**Selfie / CNIC honesty**	P1	Either read the stored selfie back into a review queue before granting the badge, or rename the badge to what the system actually checks. Same question for the CNIC heuristic. Needs a client decision; either way, state the limit in writing at handover.	No badge in the app claims more than the system actually verifies.	0.5
C · MONEY — THE ONE V1 PAYWALL						
9	☐	**Explore+ billing**	P0	Wire the paywall to real billing — Play Billing + StoreKit through RevenueCat is the fastest defensible route; a local gateway (JazzCash / Easypaisa) is the alternative if the client prefers PKR rails. Receipt validated server-side in an Edge Function that writes the subscription state. **Client decision required before this can start.**	A real purchase on a real device unlocks Explore+; cancelling revokes it.	1.5
10	☐	**Server-side entitlements**	P1	Move the 15/day cap into the `like_profile` RPC so the count cannot be edited by the client. Lock `is_explore_plus`, `subscription_plan` and `subscription_renews_at` against direct user writes so only the verified-purchase function may set them.	A hand-crafted API call cannot grant itself Explore+ or extra likes.	1
D · REACH, REAL CONTENT, COMPLIANCE						
11	☐	**Push notifications**	P1	`expo-notifications` is not installed. Add it, plus a `push_tokens` table and an Edge Function that sends on new match / new message / new like. Every send honours the existing `notification_prefs` row, which is fully built and stored but currently drives nothing outside the in-app feed.	A message to a closed app arrives as a push, and turning the pref off stops it.	1
12	☐	**Real content only**	P1	Put the mock deck top-up behind a dev-only flag so production never shows fabricated members. Replace the hardcoded Explore "Events" (`mockEvents`) with a real table, or hide the tab for V1. Write honest empty states for a thin early user base.	A production build shows only real people, or an honest empty state.	1
13	☐	**Legal & store compliance**	P0	The Legal screen is a placeholder notice. Write a real Privacy Policy and Terms covering CNIC collection, photo storage, retention and deletion; host them and link them in-app. 18+ age gate. Play Data Safety form and App Store privacy labels. Confirm the built-in account-deletion path meets both stores' rules.	Both store review checklists pass on paper.	1
E · COMPLETENESS, HARDENING, HANDOVER						
14	☐	**Emoji reactions in chat**	P2	The roadmap specifies "text chat with basic emoji reactions"; no reaction storage or UI exists anywhere in the repo. Add a `message_reactions` table and a long-press picker on `MessageBubble`.	Both sides can react and each sees the other's reaction live.	0.5
15	☐	**Chat completeness**	P2	History pagination so a long thread does not load whole, delivered / read state, and a check that voice notes and image messages survive the new schema.	A 500-message thread opens instantly.	0.5
16	☐	**Urdu / RTL & a11y sweep**	P2	Close the remaining English fallbacks in `ur.json`, walk every screen in RTL, add accessibility labels.	No untranslated string and no broken RTL layout anywhere.	0.5

#		TASK	PRI	WHAT HAS TO BE DONE	DONE WHEN	EST.
17	☐	**Performance & error states**	P2	Deck pagination instead of fetching every profile, image caching, offline and error states on every network call.	The deck scrolls smoothly at 500+ profiles; no unhandled network error.	1
18	☐	**Tests, QA pass & handover**	P2	The repository has **zero test files**. Add a smoke suite over the services layer plus CI. Then two physical devices and two real accounts through every flow end to end: signup → match → chat → move to rishta → block → report → subscribe → delete account. Production EAS builds, store listings, and a handover pack (SQL in order, env vars, runbook, written statement of known limits).	The full journey passes twice with no P0/P1 open, and the client can run the project without us.	3

> **Roughly 18.5 working days remain for one developer**
>
> Order matters more than the total. Items 1–4 must come first: reporting, blocking, push notifications, entitlements and the whole QA pass all sit on top of two real users being able to reach each other. Item 9 (billing) needs a client decision — RevenueCat vs. a local PKR gateway — before it can start, so raise that question on day one rather than when you arrive at it.

EXPLICITLY OUT OF SCOPE FOR V1

Per the roadmap these belong to V2 and should **not** take development time now, even where a screen already exists in the branch (`wali-dashboard.tsx` , `cnic-verification.tsx` , `call.tsx`): AI compatibility scoring, Family Account Mode, Wali Dashboard, bureau verification, Nikah Journey, Overseas Mode, voice and video calling, and the full 6-tier pricing structure. The only decision needed per screen is whether to hide it behind a flag for V1 or leave it visible as a preview.